

Chapter 9 - Multiple files and scope

Multiple files

With multiple files our code will look more organized and easier to navigate. Create a new file called `example.lua`. Make sure it's in the same folder as a new and empty `main.lua`.

Inside this file, create a variable. I will put `--! file:` at the top of the codeblock to make clear in what file you have to put the code. This is only for this tutorial, it has no use (it's only commentary after all) and you don't need to copy it. If a codeblock doesn't have this in future tutorials, it's either in `main.lua`, or the previous mentioned file.

```
--! file: example.lua
```

```
test = 20
```

Now in `main.lua`, put `print(test)`. When you run the game, you'll see that `test` equals `nil`. This is because we have to load the file first. We do this with `require`, by passing the filename as string as first argument.

```
--! file: main.lua
```

```
--Leave out the .lua
```

```
-- No need for love.load or whatever
```

```
require("example")
```

```
print(test)
```

We don't add the `".lua"` in the filename, because Lua does this for us.

You can also put the file in a subdirectory, but in that case make sure to include the whole path.

```
--With require we use . instead of /
```

```
require("path.to.example")
```

Now when you print `test`, after we loaded `example.lua`, you should see it says 20.

`test` in this case is what we call a *global variable* (or a *global* for short). It's a variable that we can use everywhere in our project. The opposite of a global variable, is a *local variable* (or *local* for short). You create a local variable by writing `local` in front of the variable name.

```
--! file: example.lua
```

```
local test = 20
```

Run the game to see test is now nil again. This is because of its *scope*.

Scope

Local variables are limited to their *scope*. In the case of test, the scope is the file example.lua. This means that test can be used everywhere inside that file, but not in other files.

If we were to create a local variable inside a *block*, like a function, if-statement or for-loop, then that would be the variable's scope.

```
--! file: example.lua
```

```
if true then
```

```
    local test = 20
```

```
end
```

```
print(test)
```

```
--Output: nil
```

test is nil, because we print it outside of its scope.

Parameters of functions are like local variables. Only existing inside the function.

To really understand how scope works, take a look at the following code:

```
--! file: main.lua
```

```
test = 10
```

```
require("example")
```

```
print(test)
```

```
--Output: 10
```

```
--! file: example.lua
```

```
local test = 20
```

```
function some_function(test)
```

```
if true then
```

```
    local test = 40
```

```
    print(test)
```

```
    --Output: 40
```

```
end
```

```
print(test)
```

```
--Output: 30
```

```
end
```

```
some_function(30)
```

```
print(test)
```

```
--Output: 20
```

If you run the game, it should print: 40, 30, 20, 10. Let's take a look at this code step by step.

First we create the variable test in main.lua, but before we print it we require example.lua.

In example.lua we create a local variable test, which does not affect the global variable in main.lua. Meaning that the value we give to this local variable we create is not given to the global variable.

We create a function called some_function(test) and then call that function.

Inside that function the parameter test does not affect the local variable we created earlier.

Inside the if-statement we create another local variable called test, which does not affect the parameter test.

The first print is inside the if-statement, and it's 40. After the if-statement we print test again, and now it's 30, which is what we passed as argument. The parameter test was not affected by the test inside the if-statement. Inside the if-statement the local variable took priority over the parameter.

Outside of the function we also print test. This time it's 20. The test created at the start of example.lua was not affected by the test inside the function.

And finally we print test in main.lua, and it's 10. The global variable was not affected by the local variables inside example.lua.

I made a visualization of the scope of each test to make it even more clear:

```
--! file: main.lua
test = 10
require("example")
print(test)
--Output: 10

--! file: example.lua
local test = 20

function some_function(test)
  if true then
    local test = 40
    print(test)
    --Output: 40
  end
  print(test)
  --Output: 30
end

some_function(30)

print(test)
--Output: 20
```

When creating a local variable, you don't have to assign a value right away.

local test

test = 20

Returning a value

If you add a return statement at the top scope of a file (so not in any function) it will be returned when you use require to load the file.

So for example:

```
--! file: example.lua
```

```
local test = 99
```

```
return test
```

```
--! file: main.lua
```

```
local hello = require "example"
```

```
print(hello)
```

```
--Output: 99
```

When and why locals?

The best practice is to always use local variables. The main reason for this is that with globals you're more likely to make mistakes. You might accidentally use the same variable twice at different locations, changing the variable to something at location 1 where it won't make sense to have that value at location 2. If you're going to use a variable only in a certain scope then make it local.

In the previous chapter we made a function that creates rectangles. In this function we could have made the variable `rect` local, since we only use it in that function. We still use that rectangle outside the function, but we access it from the table `listOfRectangles` to which we add it.

We don't make `listOfRectangles` local because we use it in multiple functions.

```
function love.load()
```

```
    listOfRectangles = {}
```

```
end
```

```
function createRect()
```

```
    local rect = {}
```

```
    rect.x = 100
```

```
    rect.y = 100
```

```
    rect.width = 70
```

```
    rect.height = 90
```

```
    rect.speed = 100
```

```
-- Put the new rectangle in the list
```

```
table.insert(listOfRectangles, rect)
```

```
end
```

Though we could still make it local by creating the variable outside of the love.load function.

```
-- By declaring it here we can access it everywhere in this file.
```

```
local listOfRectangles = {}
```

```
function love.load()
```

```
-- It's empty so we could remove this function now
```

```
end
```

So are there moments when it is okay to use globals? People have mixed opinions on this. Some people will tell you never to use globals. I'll tell you that it's fine, especially as a beginner, to use global variables when you need them in multiple files. Preferably these are variables that you don't plan to change after creating them (reassignment). Similarly to how love is a global variable that never changes.

Note that throughout this tutorial I use a lot of globals, but this is to make the code smaller and easier to explain.

Summary

With require we can load other lua-files. When you create a variable you can use it in all files. Unless you create a local variable, which is limited to its scope. Local variables do not affect variables with the same name outside of their scope. Always try use local variables over global variables, as they are faster.